



Nº	NOME	E-mail	Telefone
248037	Bruno Bertolini Maluf	brubemal@gmail.com	(15)99799-9909
235537	Bruno Carvalho de Oliveira	<u>brunocarvalho.ksc@gmail.com</u>	(15)99192-4104
248245	Guilherme Januário Serafim	<u>guilhermeserafim.facul@gmail.com</u>	(15)99855-4581
248224	Mateus Pasquotto Lulia Bellotto	<u>Mateusplb27@gmail.com</u>	(15)99820-8674
248312	Rodrigo Lopes de Oliveira Cardoso	<u>lopesrodrigo942@gmail.com</u>	(15)98807-4815

ATIVIDADE PRÁTICA:

(v1.0.0)

SOROCABA, SP

2026
SUMÁRIO

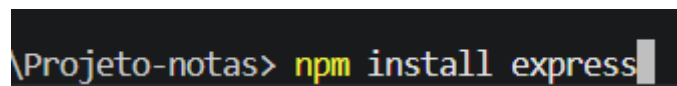
1 CONCEITO DE VANILLA JS.....	2
2 VERSIONAMENTO E DEPLOY	5
3 GESTÃO DE PROJETO.....	7
4 PROJETO.....	8
REFERÊNCIAS.....	9

1. 1 API ONLINE

Como criar uma API para usar CRUD das nossas anotações? Primeiro criamos uma API:

1. Criar a pasta do projeto
 - 1.1. Crie uma pasta chamada "projeto-notas" e abrir no VS Code.
2. Instalar o Express.js
 - 2.1. Executar a instalação no terminal: `npm install express`

Imagem 1 – Instalar Express.

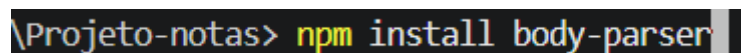


```
\Projeto-notas> npm install express
```

Fonte: Próprio Autor 2026.

3. Instalar a biblioteca Body-Parser
 - 3.1. Executar a instalação no terminal: `npm install body-parser`

Imagem 2 – Instalar body-parser.

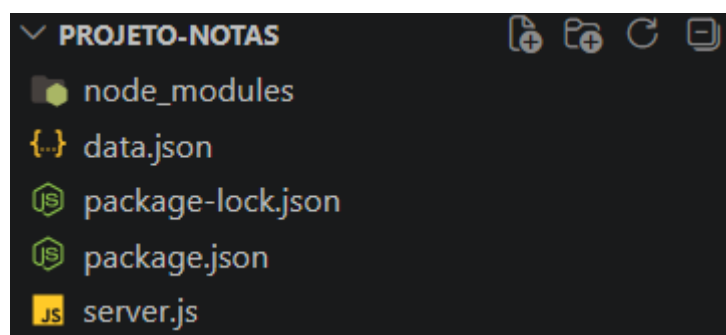


```
\Projeto-notas> npm install body-parser
```

Fonte: Próprio Autor 2026.

4. Criar um arquivo principal para armazenar a API
 - 4.1. Crie um arquivo chamado "server.js"

Imagem 3 – Estrutura de Arquivos.



```
▼ PROJETO-NOTAS
  node_modules
  {...} data.json
  package-lock.json
  package.json
  server.js
```

Fonte: Próprio Autor 2026.

4.2. Escreva esses códigos em “server.js”

Imagem 4 – Parte 1 da API.

```
1 // Importa o Express, Body-Parser e FS
2 const express = require('express');
3 const bodyParser = require('body-parser');
4 const fs = require('fs');
5 const app = express();
6 const PORT = process.env.PORT || 3000;
7 const FILE = 'data.json';
8
9 // Permite receber JSON
10 app.use(bodyParser.json());
11
12 // Libera acesso externo (CORS)
13 app.use((req, res, next) => {
14   res.header('Access-Control-Allow-Origin', '*');
15   res.header('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE, OPTIONS');
16   res.header('Access-Control-Allow-Headers', 'Content-Type, Authorization');
17   if (req.method === 'OPTIONS') return res.sendStatus(204);
18   next();
19 });
20
21 // Função para ler arquivo
22 function readNotes() {
23   try {
24     const data = fs.readFileSync (FILE);
25     return JSON.parse(data);
26   } catch {
27     return [];
28   }
29 }
30
31 // Função para salvar arquivo
32 function saveNotes(notes) {
33   fs.writeFileSync (FILE, JSON.stringify(notes, null, 2));
34 }
35
36 app.get('/', (req, res) => {
37   res.redirect('/api/notes'); // ou: res.send('API de notas rodando');
38 });
```

Fonte: Próprio Autor 2026.

Imagem 5 – Parte 2 da API.

```
40 // GET - Listar notas
41 app.get('/api/notes' , (req, res) => {
42   const notes = readNotes();
43   res.json(notes);
44 });
45
46 // =====
47 // POST - Criar nota
48 // =====
49 app.post('/api/notes', (req, res) => {
50   const notes = readNotes();
51   const novaNota = {
52     id: Date.now().toString(),
53     titulo: req.body.titulo,
54     texto: req.body.texto
55   };
56
57   notes.push(novaNota);
58   saveNotes(notes);
59   res.json(novaNota);
60 });
61
62 // =====
63 // PUT - Editar nota
64 // =====
65 app.put('/api/notes/:id', (req, res) => {
66   const notes = readNotes();
67   const index = notes.findIndex(n => n.id === req.params.id);
68   if (index >= 0) {
69     notes[index].titulo = req.body.titulo;
70     notes[index].texto = req.body.texto;
71     saveNotes(notes);
72     res.json(notes[index]);
73   } else {
74     res.status(404).json({ erro: 'Nota não encontrada' });
75   }
76 });
```

Fonte: Próprio Autor 2026.

Imagem 6 – Parte 3 da API.

```

78 // =====
79 // DELETE - Excluir nota
80 // =====
81 app.delete('/api/notes/:id', (req, res) => {
82   const notes = readNotes();
83   const novasNotas = notes.filter(n => n.id !== req.params.id);
84   saveNotes(novasNotas);
85   res.json({ mensagem: 'Nota removida' });
86 });
87
88 // Inicia servidor
89 app.listen(PORT, () => {
90   console.log(`Servidor rodando em http://localhost:${PORT}`);
91 });

```

Fonte: Próprio Autor 2026.

5. Criando o arquivo de dados

- 5.1. Crie um arquivo chamado "data.json"
- 5.2. Escreva esse código em "data.json"

Imagem 7 – Dados em JSON.

```

1  [
2    {
3      "id": "1",
4      "titulo": "Lembretes",
5      "texto": "Comprar leite e pão",
6      "criadoEm": "2026-04-28T10:00:00Z"
7    },
8    {
9      "id": "2",
10     "titulo": "Tarefas do trabalho",
11     "texto": "Enviar relatório até sexta-feira",
12     "criadoEm": "2026-04-28T10:00:00Z"
13   }
14 ]

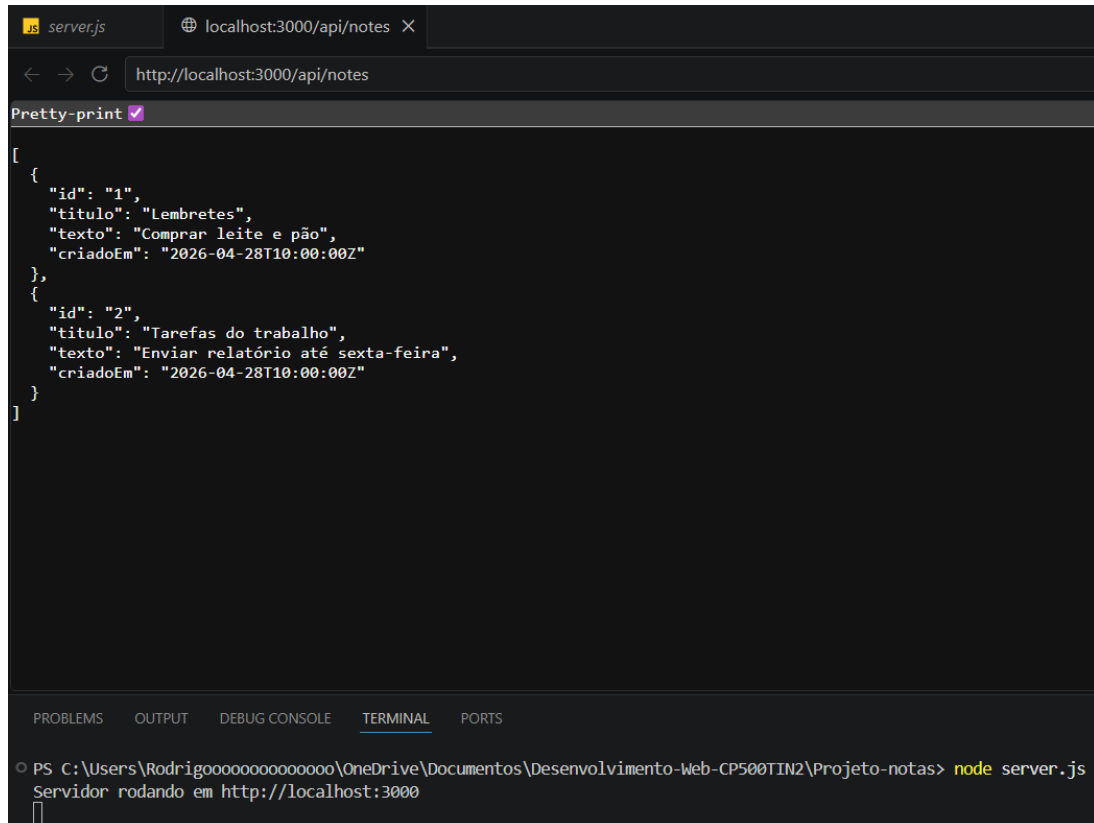
```

Fonte: Próprio Autor 2026.

6. Rodar o servidor

- 6.1. No terminal execute "node server.js" para ver a API funcionando localmente.

Imagem 8 – Dados em JSON em servidor local.



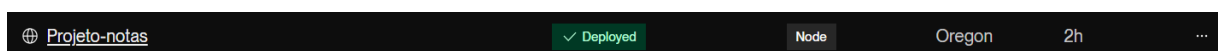
Fonte: Próprio Autor 2026.

7. Criar um Front-End em React

- 7.1. Crie uma pasta para o Front-End
- 7.2. Abra o seu terminal na pasta onde deseja salvar o projeto e execute o seguinte comando: “npm create vite@latest meu-projeto-react”
- 7.3. O terminal perguntará qual **framework**: Selecione React.
- 7.4. Depois, perguntará a **variante**: Selecione JavaScript (ou TypeScript se preferir tipagem forte).
- 7.5. O comando anterior apenas cria a estrutura de arquivos. Agora você precisa entrar na pasta e baixar os pacotes necessários: “cd meu-projeto-react” e “npm install”

8. Publique o Back-End no Render

Imagem 9 – Projeto no Render



Fonte: Próprio Autor 2026.

9. Conecte a API no React

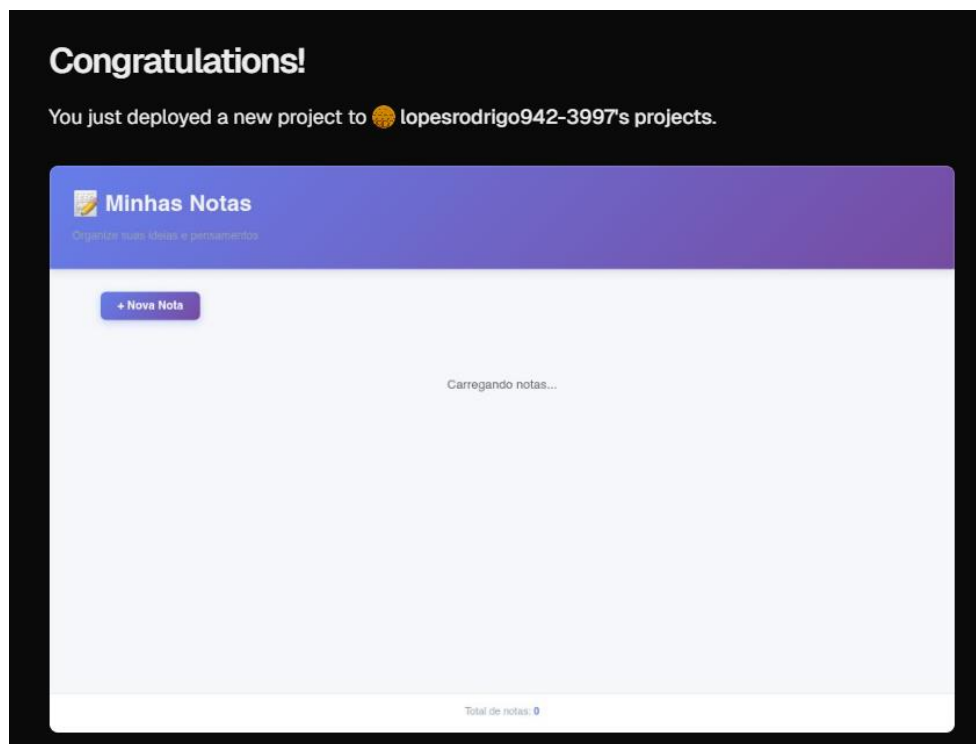
Imagem 10 – Conectar na API online.

```
const API_BASE_URL =  
  import.meta.env.VITE_API_BASE_URL ?? 'https://projeto-notas-imrd.onrender.com'
```

Fonte: Próprio Autor 2026.

10. Publique o Front-End na Vercel

Imagem 11 – Deploy na Vercel.



Fonte: Próprio Autor 2026.

2 PERGUNTAS

- **Quais são os riscos de segurança desse projeto e como poderiam ser eliminados ou mitigados?**

Os principais riscos de segurança são:

1. Ausência de autenticação e autorização, o que permite qualquer pessoa listar, editar e apagar notas.
2. Falta de validação e sanitização de entrada, o que abre espaço para XSS, payloads inválidos e corrupção de dados.
3. Exposição de endpoints sem rate limiting, o que facilita abuso e DoS;
4. Uso de arquivo local como storage, que pode vaziar dados se o servidor for comprometido ou se permissões estiverem erradas.

Isso pode ser mitigado com autenticação, controle por usuário, validação de schema, sanitização de conteúdo, CORS bem restrito, rate limiting, logs de auditoria, HTTPS obrigatório e armazenamento adequado fora do filesystem local.

- **Usar um arquivo JSON (data.json) para armazenar notas é uma boa prática em produção? Quais são as vantagens e desvantagens dessa abordagem?**

Usar um data.json em produção não é uma boa prática na maioria dos casos. A vantagem é a simplicidade: é fácil de implementar, barato e suficiente para protótipos, estudos e demos. A desvantagem é que não escala bem, não oferece concorrência robusta, não tem índice, transações, locking confiável, backup/restore eficiente nem consultas sofisticadas. Além disso, em plataformas como Vercel, Render ou containers efêmeros, o sistema de arquivos pode ser temporário, então você corre o risco de perder dados ao reiniciar ou escalar a aplicação.

- **Quais limitações um servidor baseado em arquivo JSON teria se o número de notas crescesse para 10.000 registros?**

Se o número de notas crescer para 10.000 registros, um servidor baseado em arquivo JSON começa a sofrer bastante. Cada leitura e escrita tende a envolver o arquivo inteiro, então o custo de I/O cresce junto com o tamanho do dataset. Isso aumenta latência, consumo de memória e chance de corrupção em escrita concorrente. Também fica difícil implementar paginação, busca, filtros e atualização frequente sem travar o processo. Em prática, para esse volume já

faz muito mais sentido usar um banco de dados de verdade, como PostgreSQL, MySQL ou até SQLite para projetos menores, com paginação e índices.

• **O código atual está todo em server.js. Por que isso é problemático e como você melhoraria a organização da lógica?**

O problema de colocar tudo em server.js é organizacional e técnico. O arquivo vira um monólito difícil de manter, testar e evoluir, misturando rotas, regras de negócio, acesso a dados, validação e tratamento de erro. A melhoria mais importante é separar por responsabilidade: rotas em arquivos próprios, controladores para receber e responder requisições, serviços para regras de negócio, repositórios ou camada de acesso a dados para ler/escrever persistência, e middlewares para validação, autenticação e tratamento centralizado de erros. Isso deixa o código mais legível, testável e pronto para crescer sem virar um ponto único de falha.

3 PROJETO

- Projeto
 - GitHub: <https://github.com/recamOo/Projeto-Notas-WEB>
 - GitHub: <https://github.com/recamOo/Projeto-notas>
 - Vercel: <https://projeto-notas-web-x8vm.vercel.app/>
 - Render: <https://projeto-notas-imrd.onrender.com>

REFERÊNCIAS

- GIT. Documentação oficial. Disponível em: <https://git-scm.com/>. Acesso em: 2026.
- GITHUB. Plataforma de hospedagem de código. Disponível em: <https://github.com/>. Acesso em: 2026.
- VERCEL. Plataforma de deploy de aplicações. Disponível em: <https://vercel.com/>. Acesso em: 2026.